

Conceptual practice solutions

Lectures 1–5, including Practice Notebooks 1 and 2

Open a solution only after attempting that question. The green box is the claim. The paragraph is the reason.

1 What the framework is for

1. The input is a mathematical description of a value or a dynamical system, plus constraints taken from data: how neurons are tuned, how many there are, what the time constants are. Solving for weights is the compilation step that makes a spiking network approximate that description. It is not a claim that cortex runs a least-squares fit.

Optimal weights are the compiler's output. The input is a vector description plus biological constraints.

2. One tuned neuron says something about the stimulus, and it is silent for many stimuli that other neurons would distinguish. The value lives in the pattern across the population. Decoding reads that pattern. A single spike train does not determine the value by itself.

The represented value is a population pattern. One neuron's tuning is only one coordinate of that pattern.

2 Representation, transformation, dynamics

1. Representation is the map between a vector and population activity. Transformation is the feedforward computation of a function of that vector. Dynamics makes the vector obey a differential equation, and the connection that implements the equation comes back to the population that holds the state.

Representation stores the vector. Transformation computes $f(x)$. Dynamics needs recurrence.

2. Once a population's represented value and the desired function are chosen, the decoder and the next encoder determine that connection's weights. The next connection can be solved from the next value, without revisiting the previous weights. If the intermediate values are unknown, there is no per-connection target, and a method such as backpropagation has to adjust many layers together. That is more flexible and much harder to interpret.

Known intermediate values let each connection be solved alone. Unknown intermediates remove that target.

3 What a LIF neuron is doing

1. The leak pulls voltage toward the current and, when the current is gone, back toward rest, so a past input fades. Threshold decides that the membrane has done enough to emit a spike. The refractory hold keeps the neuron from emitting another spike during the reset, which is what caps the firing rate.

Leak fades old input. Threshold emits the spike. Refractoriness blocks an immediate second spike.

2. The equilibrium of the membrane is J . Flattening just under threshold means that equilibrium is at or below threshold, so the voltage is approaching a line it is not allowed to cross in finite time. Extra time moves it closer to that line and never past it.

J is at or below threshold. The asymptote is the barrier, so waiting does not produce a spike.

4 Rate curves, and why they bend

1. Both are off, or nearly off, below their threshold and then rise once the current is large enough. Just above threshold the LIF curve is steep, because a small extra current shortens a long wait to threshold. At large current the LIF curve flattens: the refractory period remains after every spike, so the rate cannot pass $1/\tau_{\text{ref}}$. The ReLU keeps rising as a straight line.

Both turn on. The LIF is steep just above threshold and saturates. The ReLU does not saturate.

2. A stack of linear maps is still one linear map, so the network can only scale and add. A threshold, a saturation, or a bend lets the population build functions that are not linear. The hyperbolic tangent bends, and it also produces negative outputs. A firing rate is a count of spikes and does not go below zero.

Linearity cannot create a new kind of function. A negative tanh output is not a firing rate.

5 Response curves and tuning curves

1. Rate against injected current is the response curve $G(J)$. Rate against sound intensity is a tuning curve, because the horizontal axis is the stimulus. The encoding equation is the conversion: it turns the stimulus into a current, and G turns that current into a rate.

Current axis: response curve. Stimulus axis: tuning curve. Encoding connects them.

2. A falling scalar tuning curve is a negative slope from stimulus to current. In this course that sign usually sits in the encoder $e = -1$, with the gain kept positive. A peak at one preferred stimulus is not a one-dimensional slope. It appears when the stimulus is a direction, and the current is large only when that direction lines up with the encoder.

A falling scalar curve is a negative encoder. A single peak is alignment with a preferred direction.

6 Encoders, intercepts, and the radius

1. The positive encoder fires when x is above ξ . The negative encoder fires when x is below $-\xi$. They are on opposite ends of the line, with a gap around zero if ξ is positive. In that gap, for a pair with this intercept, neither is past its own threshold.

$e = +1$ fires for $x > \xi$. $e = -1$ fires for $x < -\xi$.

2. Encoders, intercepts, maximum rates, and decoders were chosen for stimuli inside the ball. Outside it, neurons are driven into a region of their tuning curves that the decoder never had to match, and many sit near their maximum rate. The error grows there. LIF neurons do not contain a mechanism that cuts the represented value off at the radius, so the failure is a bad decode, not a guaranteed hard clip.

Outside the ball the decode is an extrapolation. It need not clip.

7 A linear readout of a nonlinear code

1. If several neurons carry overlapping information, corrupting a few of them changes the weighted sum a little. A code that places the entire value in one neuron loses the value when that neuron is corrupted. The representation is still lossy because the activities are a nonlinear function of x with a limited range, and a finite linear readout cannot undo that perfectly.

Overlap tolerates lost neurons. Finite nonlinear tuning still prevents a perfect inverse.

2. Each opposing ReLU is a half-line, and subtracting them rebuilds the whole line. A LIF tuning curve bends: it is steep near threshold and flattens at high rate. A linear combination of bends is still bent, unless enough differently placed curves are averaged to iron the bend out. With a finite random population some curvature remains. That remainder is static distortion.

ReLU's here are pieces of a straight line. LIF curves are bent, so a finite linear readout cannot cancel the bend completely.

8 Noise, and why decoders shrink

1. The decoded value multiplies each activity by a weight. Independent noise in a neuron is multiplied by that same weight, so a huge weight turns a small fluctuation into a huge fluctuation in $\hat{x}$. Unregularized fits often invent those huge weights to chase tiny differences between similar neurons. The penalty punishes large weights, so the fit accepts a slightly worse match to the clean tuning curves in exchange for weights that do not amplify noise.

Large weights amplify activity noise. Regularization shrinks the weights.

2. Biologically, spikes, vesicles, channels, and recurrent networks make the activity an unreliable copy of $G(J)$. Mathematically, if two neurons have nearly the same tuning curve then AA^T is nearly singular, and the unregularized inverse explodes or fails. Adding a multiple of the identity makes that matrix invertible and damps the explosion. One term is doing both jobs.

Biology: activity is a noisy copy of the rate curve. Algebra: the penalty keeps AA^T invertible when neurons are nearly duplicates.

9 Distortion and noise are different errors

1. Noiseless neurons leave only static distortion: the linear readout still cannot perfectly undo curved tuning. Exact tuning curves, meaning the weighted sum already equals x on the clean activities, leave

only the noise term, because the noise is still multiplied by the decoder weights.

Noiseless neurons: distortion remains. Perfect clean decode: noise error remains.

2. Squared distortion falls like $1/n^2$, faster than squared noise error, which falls like $1/n$. Extra neurons give the linear readout more curves with which to cancel curvature, and that cancellation improves quickly. Independent noise averages more slowly: variances add, so the squared error halves when the population doubles. The root-mean-square error is a square root of a squared error. In a noise-dominated population that square root turns $1/n$ into $1/\sqrt{n}$.

Distortion falls faster. Noise-limited RMSE falls like $1/\sqrt{n}$ because RMSE square-roots a $1/n$ squared error.

10 Vectors, preferred directions, and the population vector

1. The neuron is silent on the side of the line $\langle e, x \rangle = \xi$ that faces away from the encoder, including the line itself at threshold. Rate grows fastest when the stimulus moves parallel to e , because that is the direction in which the projection increases fastest. Moving sideways along the threshold line does not change the current.

Silent on and behind the line perpendicular to e at the intercept. Fastest growth is parallel to e .

2. For a broad set of encoders spread around the circle, the encoder of an active neuron points roughly toward the stimulus, so using encoders as decoders gets the direction roughly right. The lengths are the problem. An encoder was chosen as a preferred direction, not as the number that converts a firing rate back into a magnitude. The fitted decoder is solved so that the weighted sum matches both direction and length. Comparing the two fairly means normalizing the decoded vectors and looking at the angular error; the raw error makes the fitted decoder look better mostly because the lengths are right.

Encoder-as-decoder: direction roughly right, magnitude wrong. Fitted D : both.

11 Spikes are not rates

1. A spike train is a sum of instantaneous pulses. Between pulses the signal is exactly zero, so the weighted sum is zero no matter what the decoders are. The rate model had already replaced the train by a continuous count of spikes per second. The filter spreads each pulse over a short time and rebuilds a trace that a rate decoder can read.

Raw spikes are zero between events. The filter restores the rate-like trace the rate model assumed.

2. A continuous delta has area 1. On a grid of spacing Δt , area 1 requires height $1/\Delta t$. A stored 1 is short by that factor, so the filtered trace comes out in spikes per bin rather than spikes per second. The rate decoder then sees a number about Δt times too small. Putting the factor on the spikes and again on the output multiplies by $1/\Delta t$ twice and makes the trace too large by the same factor.

Height 1 is short by $1/\Delta t$. The factor belongs in exactly one of: spikes, kernel, decoded output.

12 Smooth signals and their spectra

1. Independent samples jump from one time step to the next, which takes infinite frequency as the time step shrinks, and physical stimuli do not do that. They are smooth. Zeroing coefficients past a limit removes those jumps. The trace can wiggle only as fast as the highest remaining frequency.

Independent samples are unrealistically jagged. A band limit forces the trace to be smooth on the scale $1/f_{\text{limit}}$.

2. Equal RMS means the typical size of the two traces matches. The higher band limit contains faster coefficients, so that trace wiggles faster. Widening the band and then rescaling to the same RMS spreads the same power over more frequencies. The average $|X|$ plateau gets wider and lower. One individual draw still looks ragged, because each coefficient was an independent random number. The flat shape appears only after averaging many draws.

Same typical amplitude, faster wiggles. The average plateau gets wider and shorter. One spectrum stays ragged.

13 Why the optimal filter is a low-pass

1. High-frequency response power is the sharpness of the spikes, plus noise that is not a copy of the input. The numerator averages the product of that response with the input. Past the cutoff the input coefficient is zero, and uncorrelated noise does not build a consistent product, so the average is zero and H shuts those frequencies off. The decoded output is the response multiplied by H , so it keeps the input's band and loses the broadband tails.

High-frequency response power is uncorrelated with the input. H is zero there, and so is the decoded output.

2. A nonzero $h(t)$ for $t < 0$ means the estimate at time t uses spikes that occur after t . A recording can be filtered after it is stored, so those spikes are available and the filter is a legitimate offline decoder. A synapse only conducts current after a spike has arrived. The online replacement is a causal postsynaptic kernel.

Acausal means the kernel looks ahead. Fine offline. A synapse is the causal replacement.

14 Synaptic filters

1. n counts factors of t multiplying the exponential. It does not set the input's frequency. At the spike, $n = 0$ is already at its maximum and then only decays. For $n = 1$ the factor of t holds the kernel at zero at the spike; it rises and peaks one time constant later. The $n = 0$ peak is first.

n is the power of t . The $n = 0$ peak is at the spike. The $n = 1$ peak is one τ later.

2. A larger τ averages each spike over a longer window, so rapid jitter in the decode shrinks. The same widening is a lower cutoff: fast pieces of the signal are delayed and reduced. Those are the two changes.

A 2 Hz input places its power where that fixed filter still has gain near one and little phase lag, relative to the signal's own slow timescale. A 10 Hz input sits farther up the rolloff, so more of the signal itself is distorted. Spike noise that the filter did not remove is still in both traces. Less distortion is not the same statement as zero leftover noise.

Larger τ : less jitter, more lag and attenuation. The slow input is less distorted. Both decodes can still be noisy.

15 Weights are the decoder and the encoder folded together

1. Decoding produces a low-dimensional value. Encoding turns that value into a current in each postsynaptic neuron, using that neuron's gain and encoder. Substituting the decoded value into the current leaves a coefficient in front of each presynaptic rate. That coefficient is the synapse. Nothing in the tissue computes the decoded value as a separate step and then re-encodes it.

w_{ij} is the decoder of neuron j passed through the encoder of neuron i . The intermediate value is not a biological stage.

2. The full matrix is one weight per pair of neurons. Decoding and re-encoding only touch the low-dimensional value, so the work scales with the number of neurons times the dimension, not with the product of the two population sizes. Simulators keep the factored form for that reason. Dale's principle says the outgoing synapses of one cell share a sign, because the cell releases one kind of transmitter. A row or column of ED is generally a mixture of positive and negative numbers. The algebra can be right while the signs are not a legal neuron.

Factored decode-and-encode is cheaper because the dimension is small. ED usually gives one presynaptic neuron both signs, against Dale's principle.

16 Decoding a function, not the identity

1. Doubling the identity decoder doubles the reconstructed value, which is $2x$. The $+1$ is a second target, the constant function. Least squares is linear in the target, so the decoder for $2x + 1$ is the doubled identity decoder plus the decoder whose training target was the constant 1. Leaving that second piece out shifts the whole reconstruction.

$2D$ reconstructs $2x$. The constant needs its own decoder, and the two decoders add.

2. The smooth target is closer. Each tuning curve is a smooth function of the stimulus, and a weighted sum of smooth curves is smooth. A jump is not in that family. The population can approximate it, and the approximation gets better with more neurons, but the same curves match a smooth target with less leftover error.

Smooth targets match smooth tuning curves. Jumps are a worse match for the same population.

17 More than one incoming population

1. Currents add, and $3y - x$ is already a function of y plus a function of x . One connection decodes

$3y$, the other decodes $-x$, and the postsynaptic current is their sum. The product xy changes with x by an amount that depends on y . No split into “something about x alone, plus something about y alone” does that for every pair (x, y) . Both variables have to drive the same tuning curves before a decoder can see the product.

A sum of single-variable functions is two connections. A product needs one population that represents (x, y) .

2. $(x + y)^2 = x^2 + 2xy + y^2$. The sum of the separate squares is missing $2xy$. Decode x and y into one scalar population that represents the sum, then decode the square of that scalar into the output. The nonlinearity now has one input, so it never needs a two-dimensional population. A joint population that decodes $(x + y)^2$ directly is also legal. It is not required.

The cross term $2xy$ is missing. Sum first, then square the scalar sum.

18 What a chain adds, even when the decoder is right

1. Each synapse is a filter with a delay of about τ , even when the weights compute the right function. Three stages put about 3τ of lag between the input and the output. The output is a delayed copy, not a simultaneous one.

Expect about three synaptic time constants of lag. Perfect decoders do not remove it.

2. Filters begin at zero, so at the start of the trial the trace has not yet charged up to the value. That is the startup transient. After it catches up, the remaining jaggedness is spike ripple: each spike is a bump in the filtered trace. The radius has been ruled out because the value stayed where the decoders were fitted. Lag can be part of the same picture, and it is not required to explain a trace that is low at the start and jagged afterward.

Startup, then spike ripple. The radius is not the cause.

The original practice test asks for the same mechanisms with the idea boxes removed. Name the mechanism first. The sketch or the matrix is the mechanism drawn in the notation of the question.